PARALLEL COMPUTING

FIT3143
LAB 03

Shee Seng Cheng (34612467) - sshe0113@student.monash.edu
Tay Chee Hsian (34612513) - ctay0040@student.monash.edu

TASK 1

EV CHARGING NODES

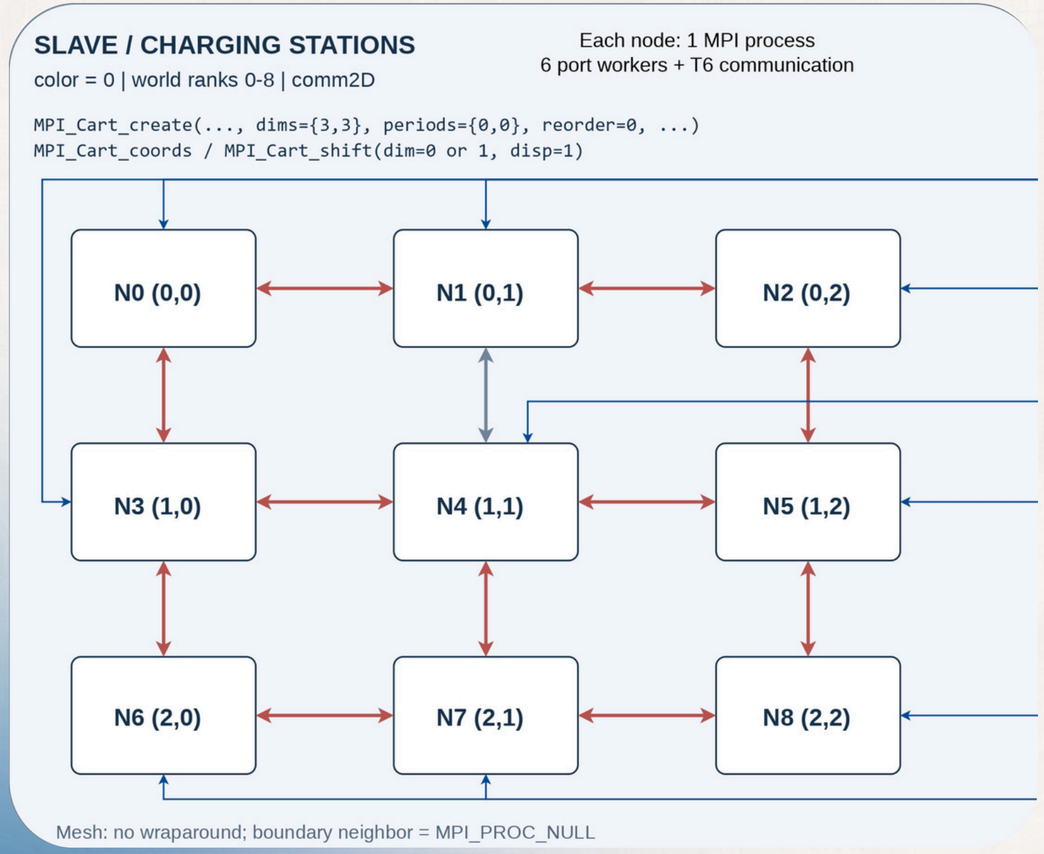
01
Architecture & Simulation Model

02
Message Passing

03
Graceful Shutdown

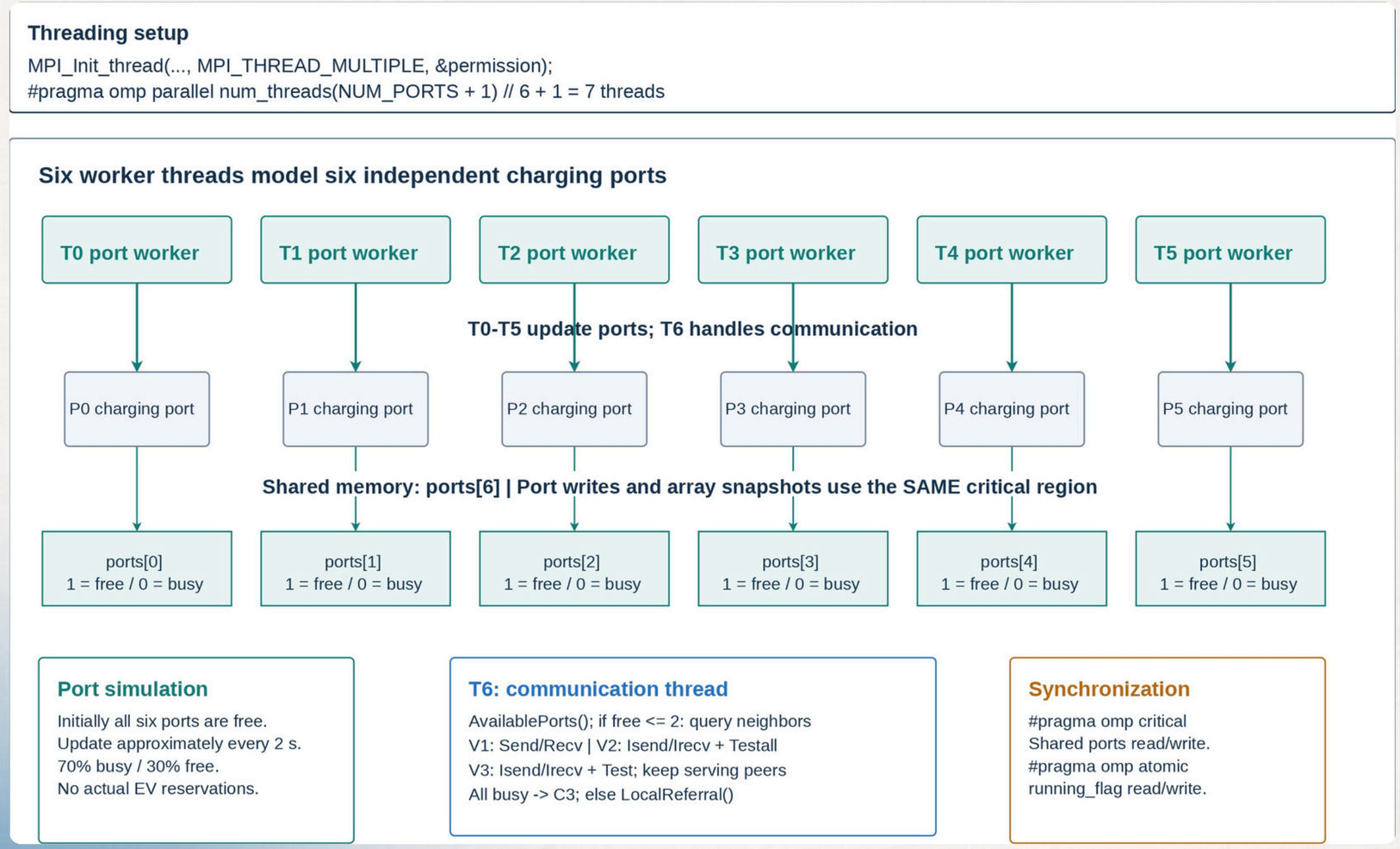
MPI Virtual Topology

```
MPI_Comm_split(MPI_COMM_WORLD, rank == size - 1, 0, &new_comm);
MPI_Cart_create(comm, 2, dims, wrapAround, 0, &comm2D);
MPI_Cart_coords(comm2D, global_rank, 2, coord);
MPI_Cart_shift(comm2D, SHIFT_ROW, DISP, &nbr_i_lo, &nbr_i_hi);
MPI_Cart_shift(comm2D, SHIFT_COL, DISP, &nbr_j_lo, &nbr_j_hi);
```



- **MPI_Cart_create** dynamically maps linear 1D MPI ranks into a 2D mesh, perfectly mirroring the physical layout of the EV charging network.
- **MPI_Cart_shift** seamlessly pinpoints the immediate directional neighbors without requiring complex hardcoded math.
- Disabling periodicity (wrap_around = 0) safely handles grid boundaries, naturally **returning MPI_PROC_NULL for edge nodes** to prevent out-of-bounds communication.

EV Charging Nodes Diagram



- **6 OpenMP worker threads** autonomously simulate dynamic port usage, governed by **#pragma omp critical** to ensure safe access to the **shared ports[6] array**.
- The local ports simulators and the communication engine (T6) synchronize effortlessly via shared memory, completely bypassing internal MPI overhead.
- To prevent gridlock during neighbor availability checks, the communication engine strictly relies on a centralized Base Station token (**MSG_QUERY_GRANT**) to safely execute blocking **MPI_Send/Recv** operation.

Message Passing

ID	Route / communicator	Tag / payload	MPI calls
C1	Requester -> neighbor comm2D	MSG_REQ_NEIGHBOR | tag 1 1 MPI_INT: dummy signal req=1	V1: Send/Recv | V2: Isend/Recv V3: Isend/Irecv + Test
C2	Neighbor -> requester comm2D	MSG_RES_NEIGHBOR | tag 2 1 MPI_INT: free-port count	V1: Send/Recv | V2: Send/Irecv V2: Testall; V3: Isend/Irecv + Test
C3	Congested node -> base MPI_COMM_WORLD	MSG_REPORT | tag 3 1 report_type: MsgData	V1/V2: Send/Recv V3: Isend/Irecv + Test
C4	Base -> reporting node MPI_COMM_WORLD	MSG_BS_REPLY | tag 4 1 MPI_INT: found_count If >0: found_count*2 MPI_INT [row0,col0,row1,col1,...]	V1/V2: Send/Recv | V3: Isend/Irecv + Test Count, then optional coordinates; same tag. found_count=0: no candidate.
C5	Base <-> all stations MPI_COMM_WORLD | 1 MPI_INT each	DISABLE (6) / READY (7) TERMINATE (5) / TERMINATED (8)	V1: Send/Recv | V2: mixed V3: Isend/Irecv + Test

MsgData: one congestion report

double timestamp; char incident_time[32];
int sender_rank, coords[2], avail_ports;
int avail_neigh[4], msg_exchanged, msg_size;
avail_neigh order: top, bottom, left, right; absent = -2.

Datatype and completion

MPI_Get_address
MPI_Type_create_struct(8, ...) + MPI_Type_commit
V2: Testall(receives); Waitall(sends)
V3: Test each active request to completion

- **C1 & C2: comm2D** utilizes **MPI_Send/Recv** for neighbor availability checks, safely orchestrated by the Base Station centralized token system to prevent gridlock.
- **C3:** The **MSG_REPORT** relies on a custom **MPI_Type_create_struct** to minimize network overhead during severe congestion alerts.
- A custom **MsgData** struct delivers complete congestion context – including timestamps, 2D coordinates, neighbor statuses, and message sizes – in a single transmission.
- **C4 & C5:** The Base Station handles EV dispatch and graceful termination (**MSG_TERMINATE**) via standard **MPI_Send/Recv** on **MPI_COMM_WORLD**.

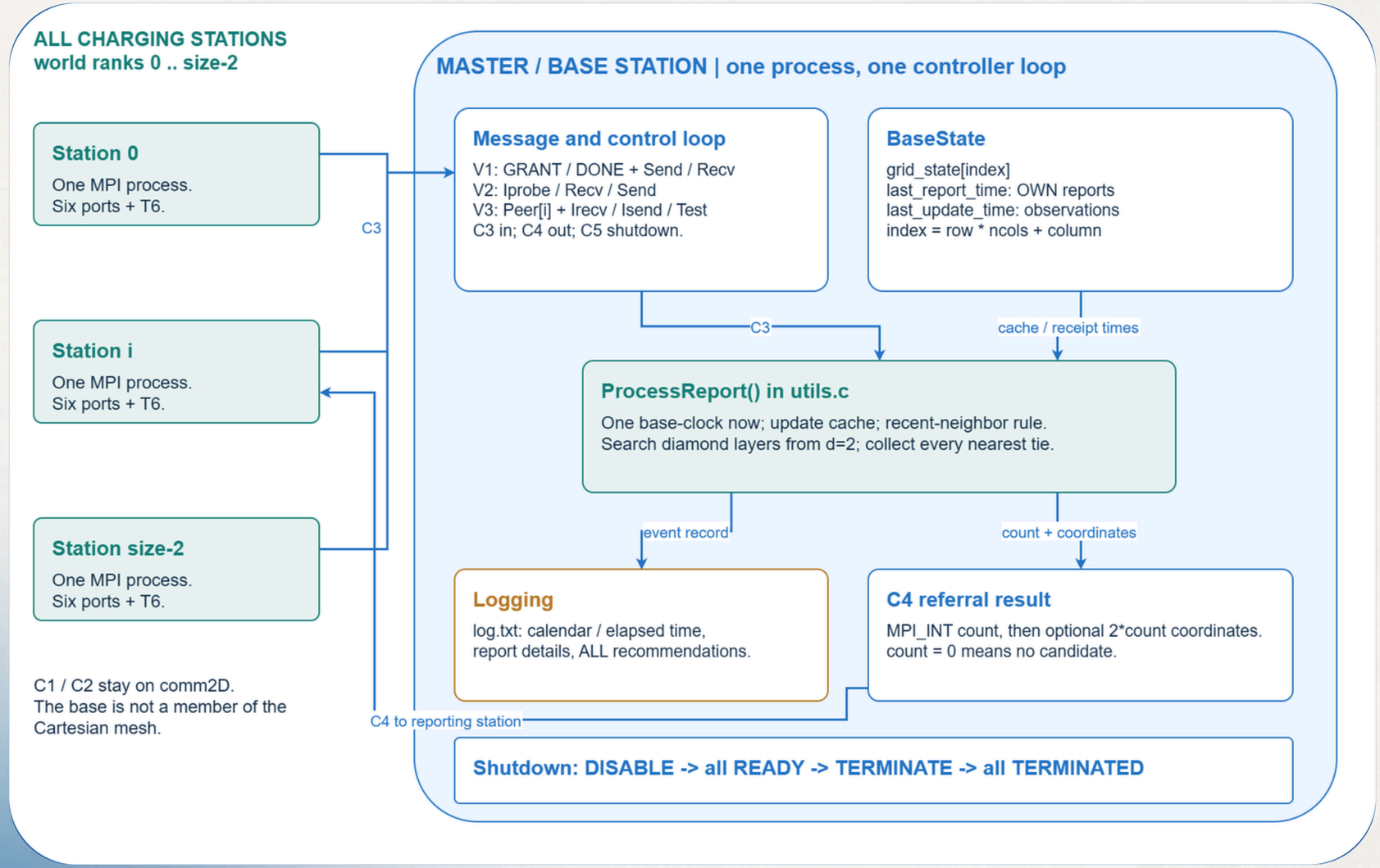
TASK 2
BASE STATION

01
Architecture & Simulation Model

02
Internal Workflow

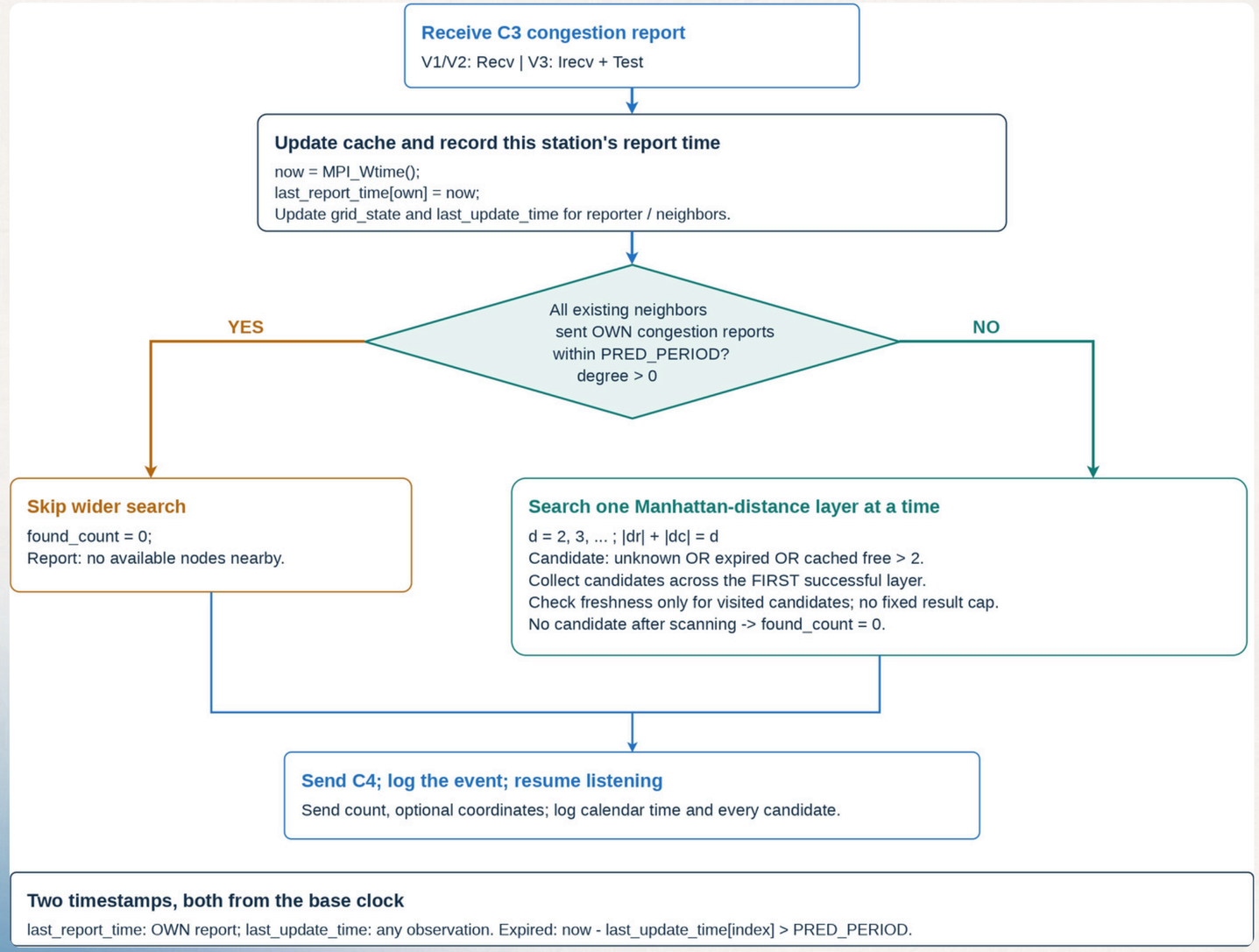
03
Predefined Period

Base Station Diagram



- Sequentially sends **MSG_DISABLE** universally upon reaching the **30 seconds simulation threshold** to halt new node inquiries.
- **Continues processing in-flight congestion alerts** while independently **tallying incoming MSG_READY** signals.
- Strictly withholds the **MSG_TERMINATE** sends until all nodes report an idle state, preventing premature network teardown.
- **MPI_Finalize** exclusively after every node returns a **MSG_TERMINATED** confirmation, guaranteeing absolute data integrity,

Base Station Workflow



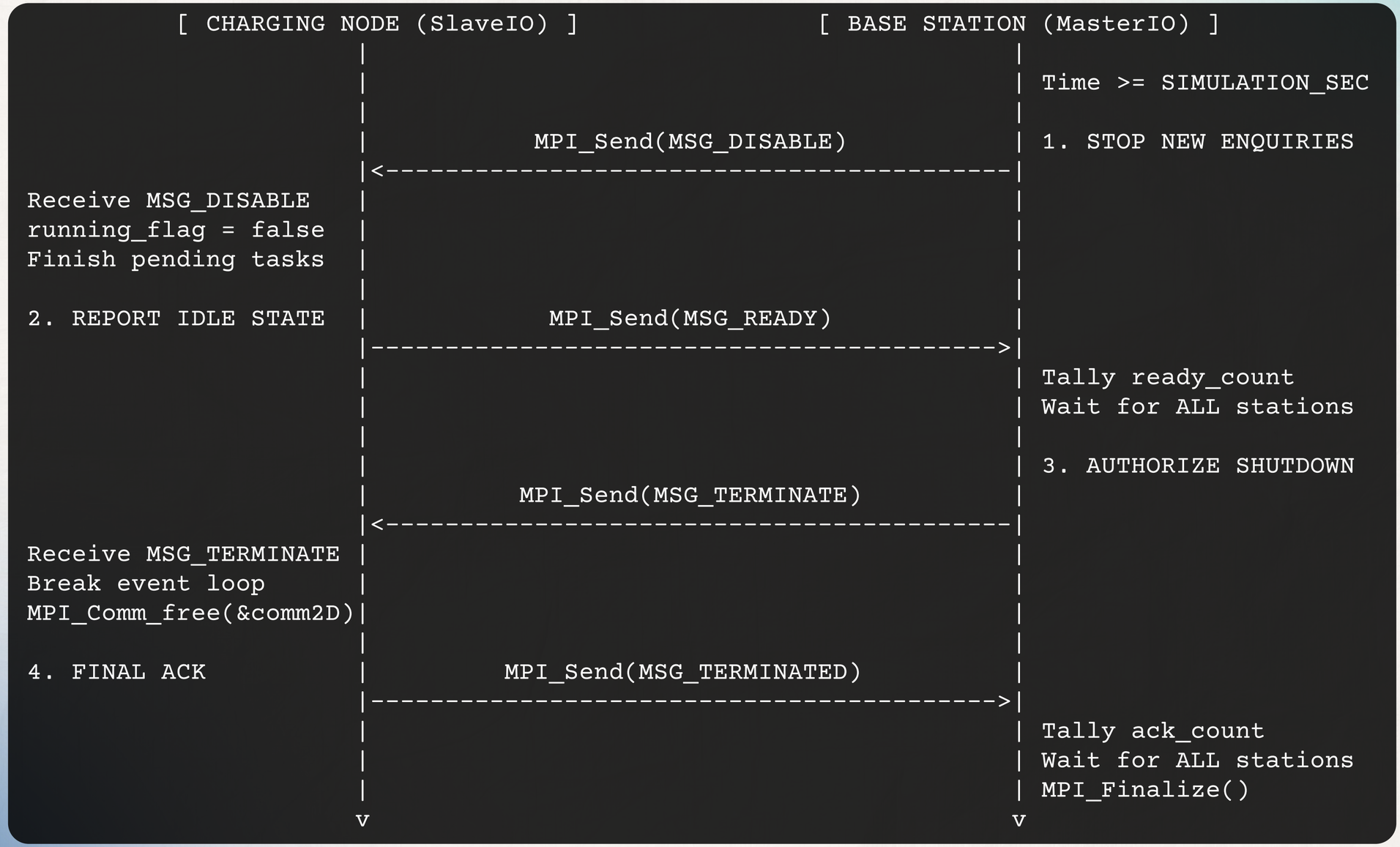
- All incoming **congestion reports** (**MSG_REPORT**) are evaluated strictly against the **Base Station's local clock** (**MPI_Wtime**), eliminating distributed timing errors across the grid.
- Before initiating a **wide-area search**, the system evaluates if all immediate neighbors have filed their own reports **within a 1.5 seconds window** (**PRED_PERIOD**). If **true**, it **skips the search** to save compute cycles.
- If required, the algorithm **expands outward in a diamond pattern**, searching exactly one **Manhattan distance layer** at a time to locate the **absolute closest available nodes**.
- The search algorithm **collects and returns all quanlifying candidates** within the first successful layer simultaneously, avoiding arbitrary or biased routing.

Predefined Period

Period (seconds)	Alerts	Shortcut	Recommended node
1.5	41	1	40
3	36	3	32
6	46	13	27

- **1.5 seconds** maximizes valid routing recommendations (40 nodes) using strictly fresh data.
- **6 seconds** Increases shortcuts (up to 13) to skip heavy searches and save CPU cycles.
- **Longer periods** rely on stale cache, **severely dropping** recommendation accuracy (40 -> 27).

Graceful Shutdown



OPTIMIZATION

01

Version 2:
Concurrent Neighbor Enquiries

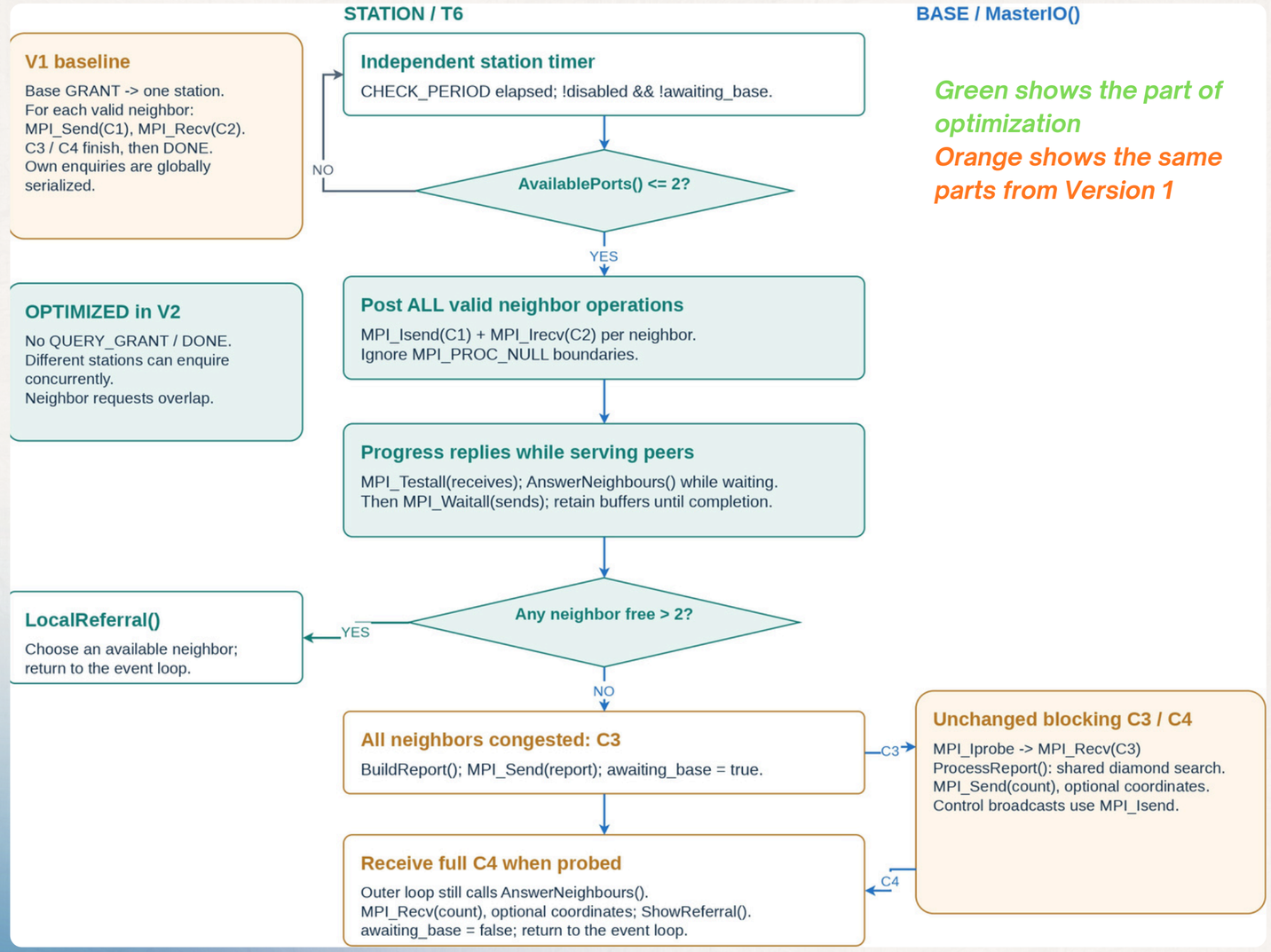
02

Version 3:
Fully Asynchronous State Machine

03

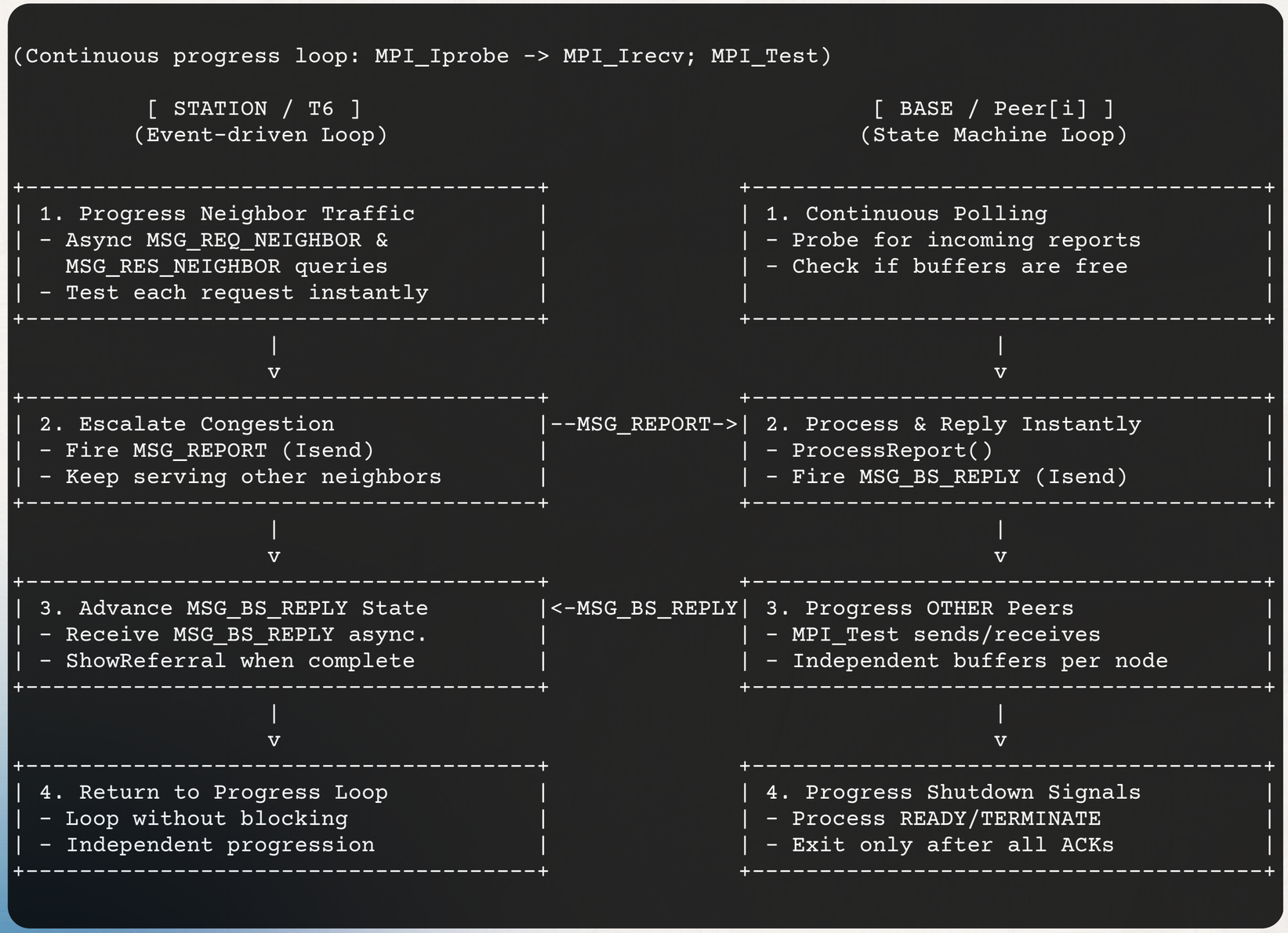
Computational Optimization:
Manhattan Diamond Search

Optimization: (V2) Concurrent Neighbor Enquiries




```c
// Overlap all neighbor requests
for (int i = 0; i < 4; i++) {
    if (neighbors[i] == MPI_PROC_NULL) continue;

    // Non-blocking Send & Receive instantly
    MPI_Isend(neighbors[i], MSG_REQ_NEIGHBOR,
              &sends[degree]);
    MPI_Irecv(neighbors[i], MSG_RES_NEIGHBOR,
              &receives[degree]);
    degree++;
}

// Progress replies while others
int done = 0;
while (!done) {
    // Check if neighbors replied, without
       blocking
    MPI_Testall(degree, receives, &done,
                MPI_STATUSES_IGNORE);

    // Answer incoming requests while waiting
    if (!done) AnswerNeighbours(...);
}

// Safely clear send buffers
MPI_Waitall(degree, sends, MPI_STATUSES_IGNORE);
```

Optimization: (V3) Fully Asynchronous State Machine




```c
typedef struct {
    MPI_Request incoming, send_count,
                send_coords;
    MsgData report;
    int count, *coords;
} Peer;

Peer peers[stations];

while (ack_count < stations) {
    for (int i = 0; i < stations; i++) {
        Complete(&peers[i].send_count);
        Complete(&peers[i].send_coords);

        if (MPI_Iprobe(i, MSG_REPORT, ...)) {
            MPI_Irecv(&peers[i].report, ...,
            &peers[i].incoming);
        }

        if (Complete(&peers[i].incoming)) {
            peers[i].count = ProcessReport(..);
            MPI_Isend(&peers[i].count, ...,
            &peers[i].send_count);

            if (peers[i].count > 0)
                MPI_Isend(peers[i].coords, ...,
                &peers[i].send_coords);
        }
    }
}
```

Optimization: Manhattan Diamond Search

7 x 7 WORKED EXAMPLE | reporter at (3,3)

(0,0) not visited	(0,1) not visited	(0,2) not visited	(0,3) 4 free	(0,4) not visited	(0,5) not visited	(0,6) not visited
(1,0) not visited	(1,1) not visited	(1,2) <=2 | d=3	(1,3) <=2 | d=2	(1,4) <=2 | d=3	(1,5) not visited	(1,6) not visited
(2,0) not visited	(2,1) <=2 | d=3	(2,2) <=2 | d=2	(2,3) <=2 | skip	(2,4) <=2 | d=2	(2,5) <=2 | d=3	(2,6) not visited
(3,0) <=2 | d=3	(3,1) <=2 | d=2	(3,2) <=2 | skip	(3,3) REPORT <=2	(3,4) <=2 | skip	(3,5) <=2 | d=2	(3,6) 3 free
(4,0) not visited	(4,1) <=2 | d=3	(4,2) <=2 | d=2	(4,3) <=2 | skip	(4,4) <=2 | d=2	(4,5) <=2 | d=3	(4,6) not visited
(5,0) not visited	(5,1) not visited	(5,2) <=2 | d=3	(5,3) <=2 | d=2	(5,4) <=2 | d=3	(5,5) not visited	(5,6) not visited
(6,0) not visited	(6,1) not visited	(6,2) not visited	(6,3) expired 0	(6,4) not visited	(6,5) not visited	(6,6) not visited

Color / search order

RED: reporting node; orange d=1 already confirmed congested.
Pale orange: visit 8 cells in d=2; blue / green: visit 12 in d=3.
GRAY: outside the first successful layer, never checked.

Return ALL three candidates at distance 3

(0,3), (3,6), (6,3) -> found_count = 3.
The expired entry is an availability estimate, not confirmed free.

0 Recent-own-report shortcut is false

At least one immediate neighbor lacks a recent OWN report.
Its C2 observation may still be congested; that does not update its OWN-report clock.

1 Start at d = 2, not d = 1

C3 has already established that valid immediate neighbors are congested.

2 Scan the complete current diamond layer

d=2: 8 fresh congested cells -> no candidate.
d=3: visit all 12 cells -> collect every qualifying tie.

3 Test freshness only for visited candidates

last_update_time[index] < 0
OR now - last_update_time[index] > PRED_PERIOD
OR grid_state[index] > THRESHOLD
All comparisons use one base-clock now.

4 Stop before d = 4

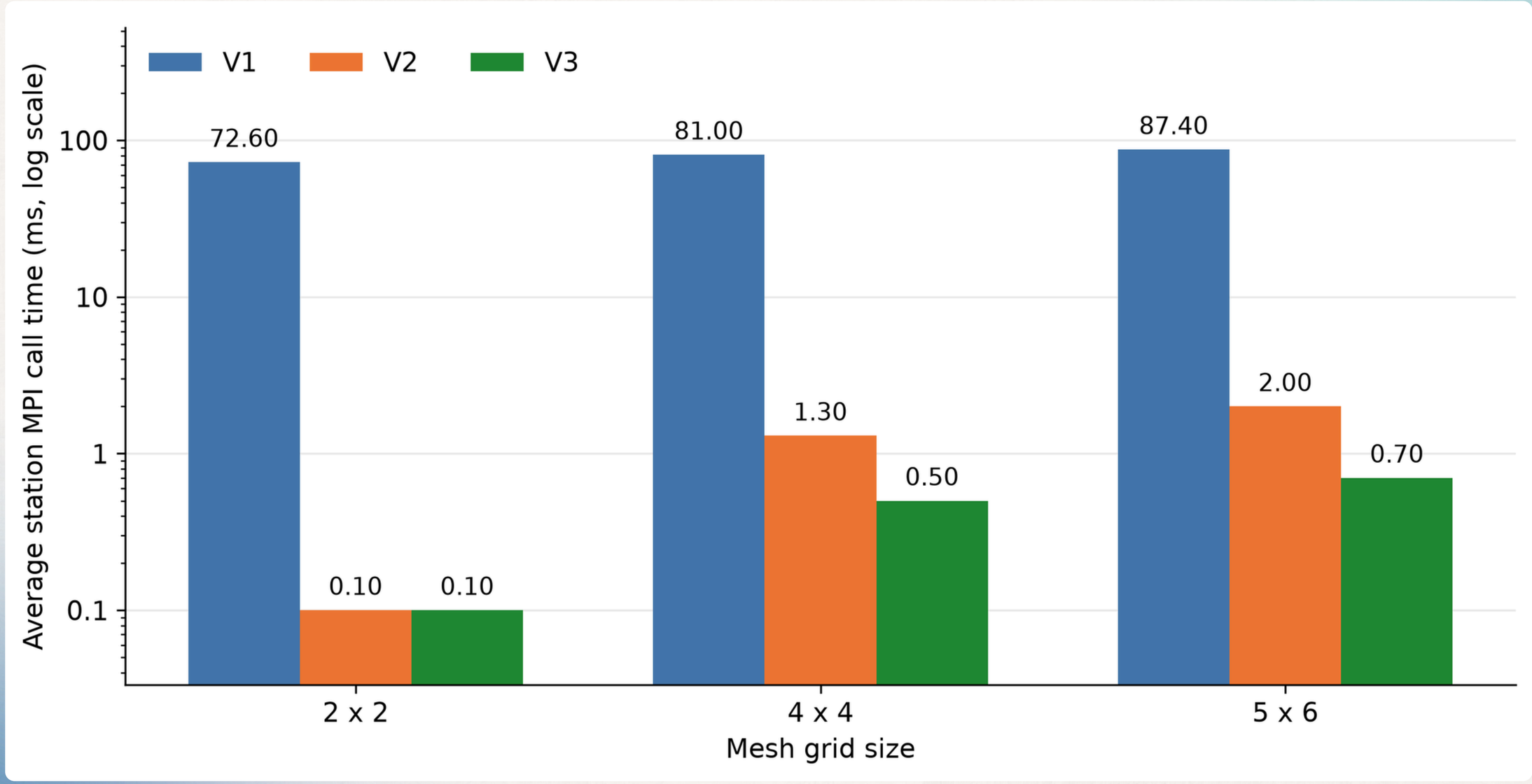
This example: 8 + 12 = 20 candidate checks, versus 49 in a full nested-loop grid scan. No full-cache expiry pass.

What is optimized / what remains unchanged

Less search work when nearby candidates exist; return every equal-nearest tie.
Worst case still O(rows * cols) if no qualifying candidate is nearby.

```
// Skip if local neighbors are recently congested
if (!(degree > 0 && recent)) {

    // 1. Start at Manhattan distance d = 2
    for (int d = 2; d <= max_distance &&
         found_count == 0; d++) {

        // 2. Scan the complete current diamond
              layer
        for (each cell where |delta_row| +
             |delta_col| == d) {

            // 3. Test freshness for visited
                  candidates
            if (is_unknown(cell) ||
                is_expired(cell) ||
                grid_state[cell] > THRESHOLD) {
                    candidates[found_count++] =
                    cell; // Collect all ties
            }
        }

        // 4. Finish this layer, then stop if
              candidates were found.
    }
}
```

Version Comparison



TASK 3 ANALYSIS

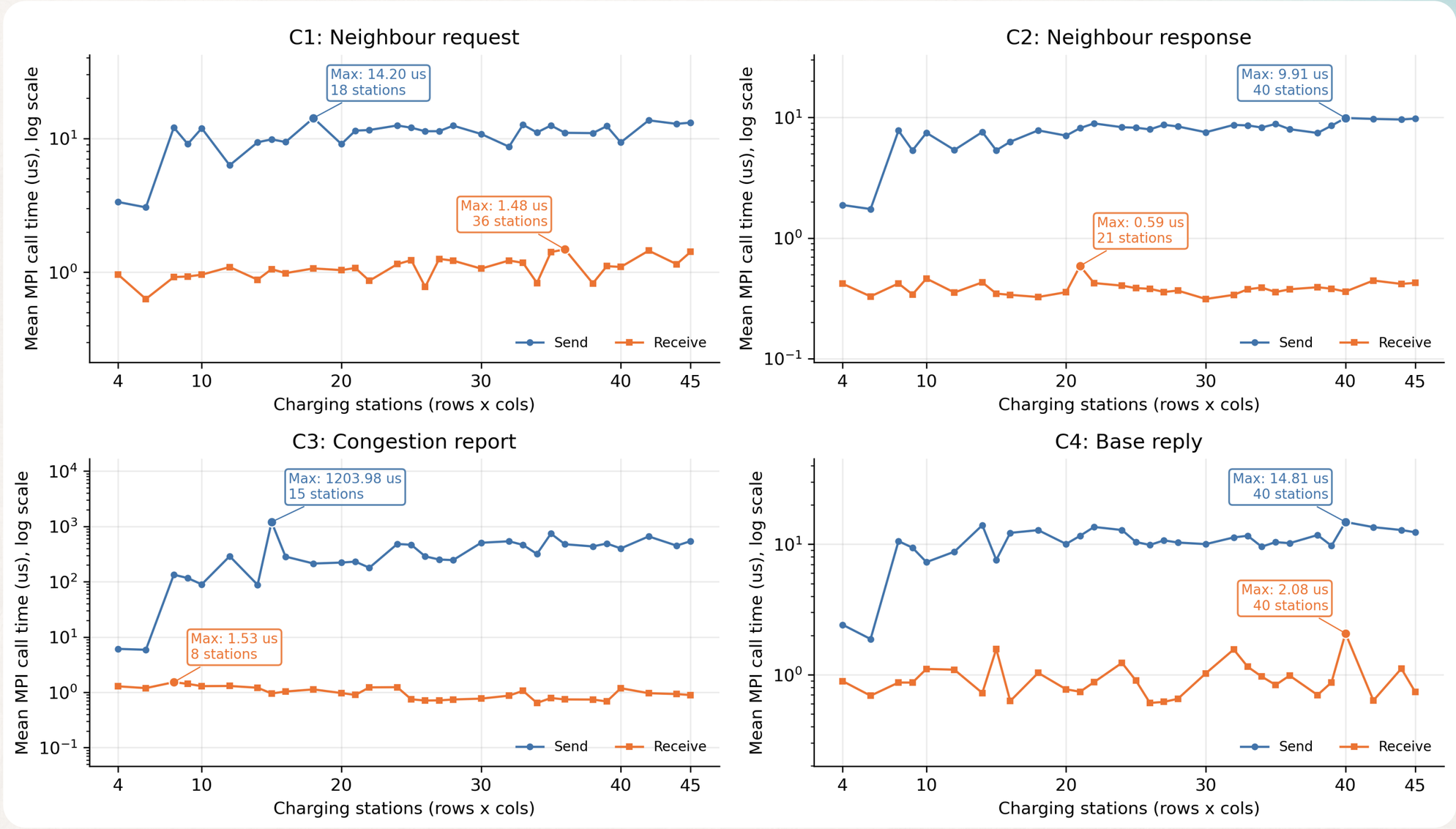
01

Increases of EV Grid Size

02

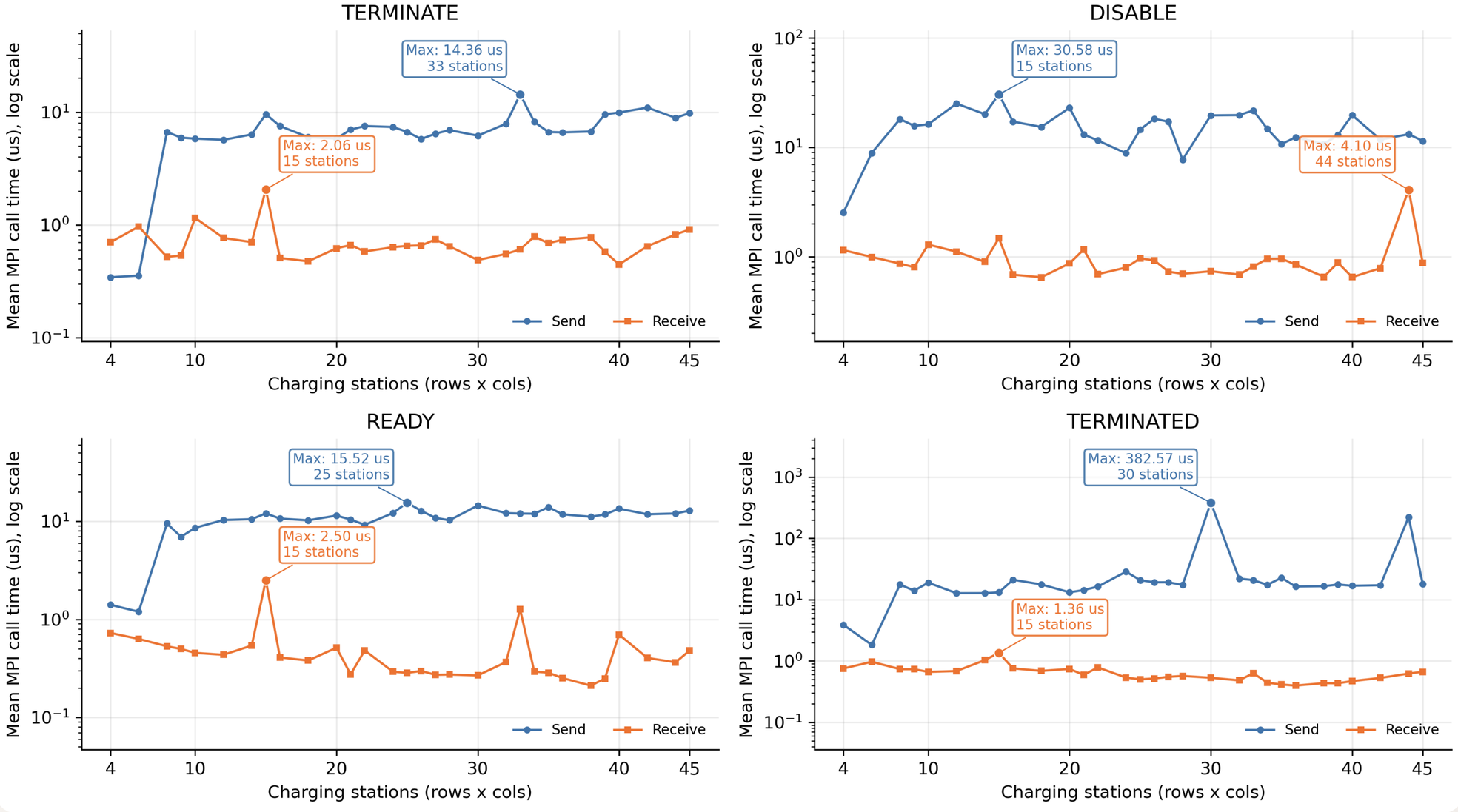
Increases of Charging Node on Line Topology

MPI COMMUNICATION TIME MESH



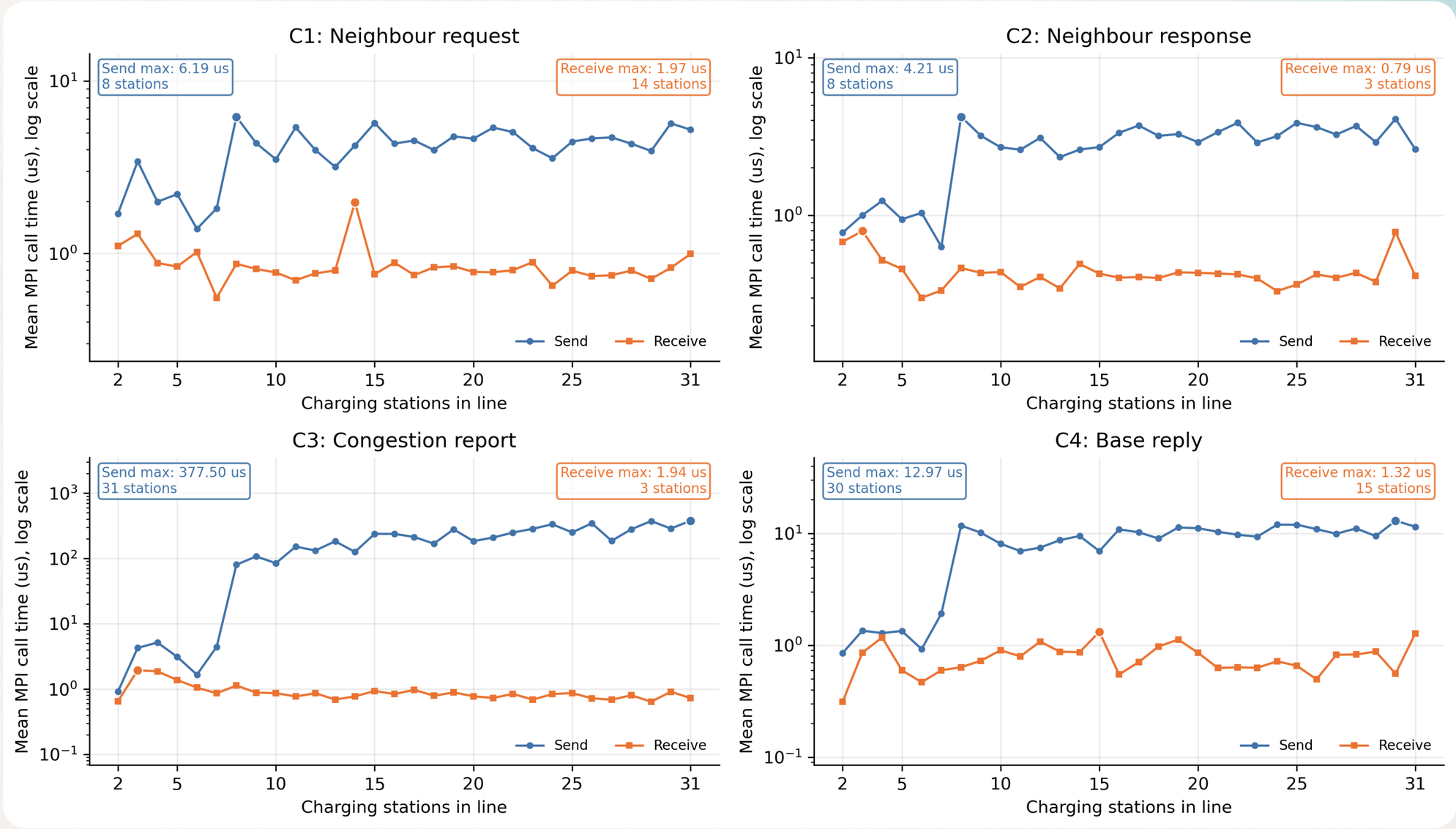
SHUTDOWN

MESH

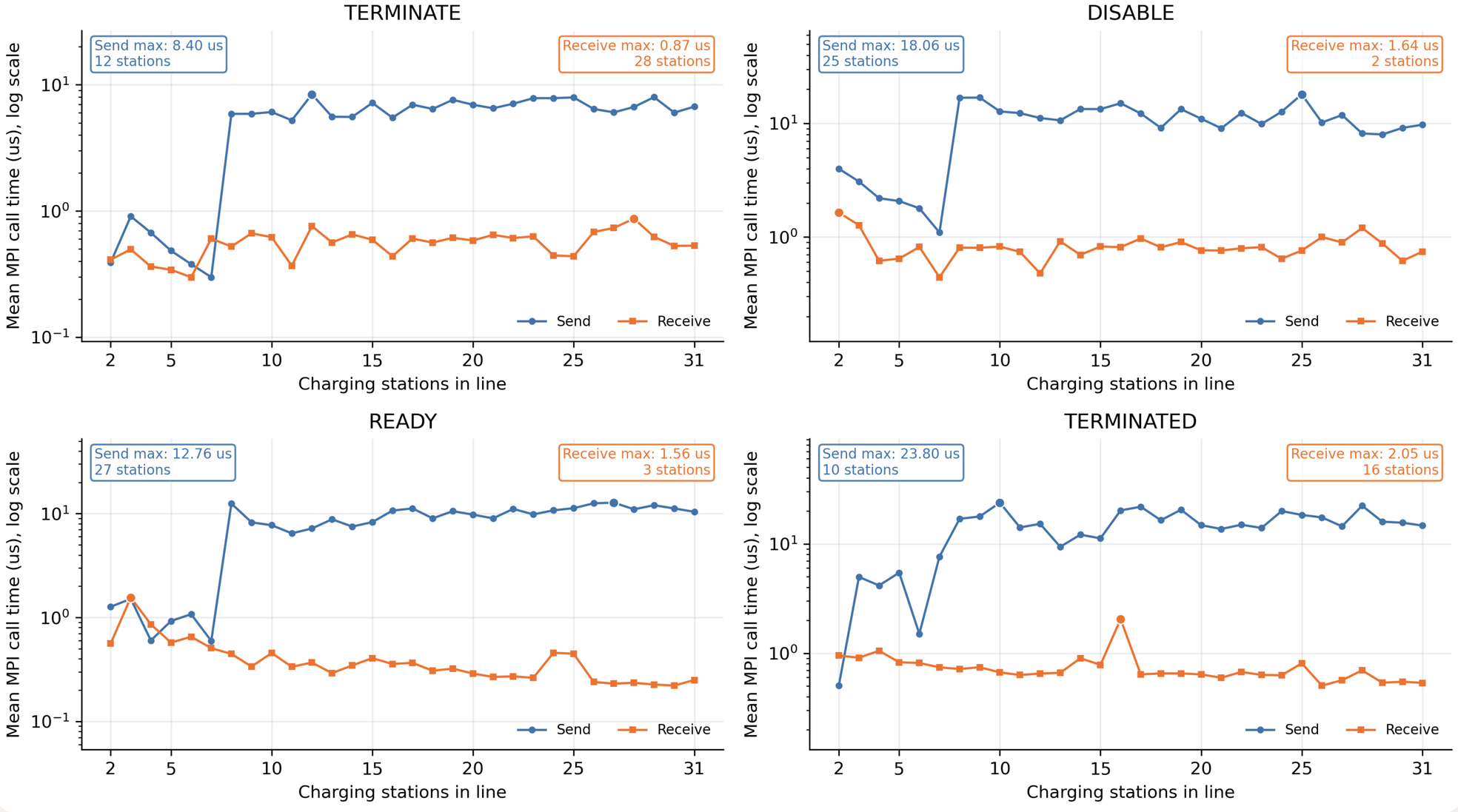


MPI COMMUNICATION TIME

LINE



SHUTDOWN LINE



CONCLUSION

Conclusion

- Combined MPI and OpenMP to simulate charging stations, neighbour enquiries, base referrals and coordinated shutdown.
- Selected V2 for overlapping communication with manageable code complexity.

Limitations

- One random run per configuration limits repeatability.
- MPI call times provide incomplete transfer-delay measurements; cached availability may become outdated.

Future Work

- Repeat experiments with controlled workloads and measure communication completion delays.
- Confirm candidate availability before referrals and test larger networks.

THANK YOU

APPENDIX

To provide a holistic view of our system's evolution, several workflow diagrams (e.g., Message Passing, Graceful Shutdown) depict the complete architecture spanning V1 to V3. However, our system is strictly modularized as follows:

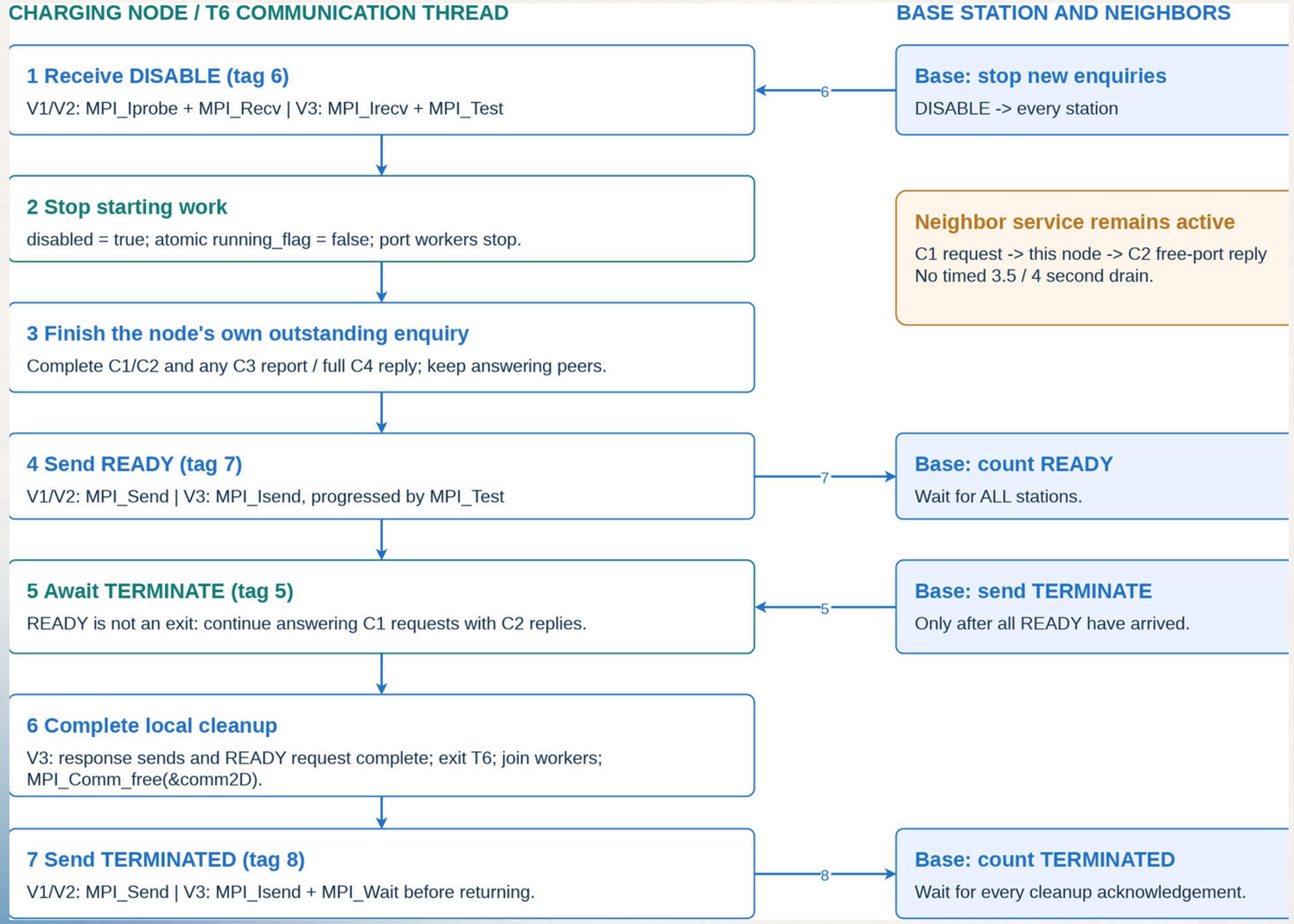
Version 1 (The Baseline):

- **Purpose:** Fulfills all core requirements for Task 1 and Task 2.
- **Mechanism:** Implements a Scheduled Blocking Architecture. It relies on centralized token passing (MSG_QUERY_GRANT) from the Base Station and standard blocking MPI_Send/Recv to guarantee absolute safety and zero deadlocks.
- *Note: When presenting Task 1 and Task 2, the V1 paths in the diagrams represent our baseline implementation.*

Version 2 & Version 3 (The Optimizations):

- **Purpose:** Exceeds baseline requirements to achieve High Distinction performance.
- **Mechanism:** Deprecates the V1 Token system. Introduces concurrent non-blocking neighbor enquiries (MPI_Isend/Irecv) and a fully asynchronous event-driven state machine.
- *Note: These are exclusively evaluated in the final 'Optimization' section of our presentation.*

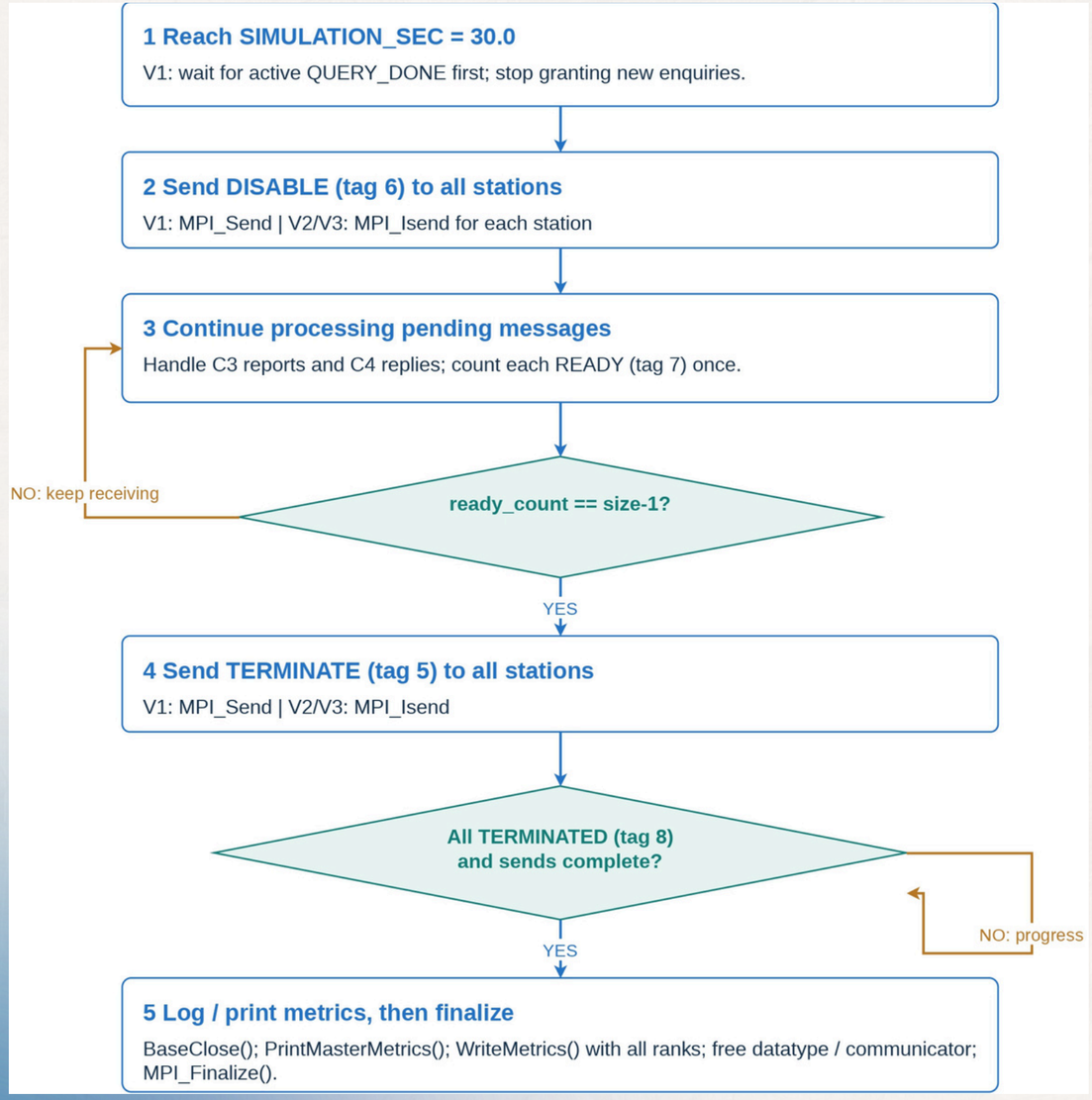
Graceful Shutdown for EV Charging Nodes




```c
while (1) {
    // 1. Probe for Base Station
          signals
    MPI_Iprobe(Base, ANY_TAG, &flag,
    &status);

    // 2 & 4. Stop work & send READY
    if (status.TAG == MSG_DISABLE) {
        running_flag = false;
        MPI_Send(Base, MSG_READY);
    }

    // 3 & 5. Flush pending C1/C2
              requests
    AnswerNeighbours(...);

    // Break only when absolutely
       safe
    if (status.TAG == MSG_TERMINATE)
        break;
}

// 6 & 7. Local cleanup & Final ACK
MPI_Comm_free(&comm2D);
MPI_Send(Base, MSG_TERMINATED);
```

Graceful Shutdown

for Base Station




```
// 1 & 2. Send DISABLE after reaching 30 sec
if (time >= 30) {
    for (int i = 0; i < stations; i++) {
        MPI_Send(Node_i, MSG_DISABLE);
    }
}

while (ack_count < stations) {
    MPI_Iprobe(&status);

    // 3. Process pending messages
    if (status.TAG == MSG_READY) ready_count++;
    if (status.TAG == MSG_TERMINATED) ack_count++;

    // 4. Send TERMINATE
    if (ready_count == stations && !terminating) {
        for (int i = 0; i < stations; i++) {
            MPI_Send(Node_i, MSG_TERMINATE);
        terminating = true;
    }
}

// 5. Finalize
BaseClose();
MPI_Finalize();
```